

# VMS

## High-Level Design (HLD)

Architecture, major workflows, integrations, security controls, deployment model, and operational design

|                |                                                               |
|----------------|---------------------------------------------------------------|
| Document owner | Engineering / VMS Platform                                    |
| Version        | 1.0                                                           |
| Prepared on    | 07 September 2026                                             |
| Classification | Internal - Confidential                                       |
| Design basis   | Repository analysis and observed Nginx Dev/Production routing |

# Document Control and Contents

This HLD describes the logical and deployment architecture of the Vendor Management Suite (VMS). It is implementation-aware but intentionally avoids source code, credentials, API secrets, and production data.

| Area                | Coverage                                                                                                                            |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Business capability | Vendor lifecycle, screening, KYC, approvals, invoices, deboarding, notifications, chat                                              |
| Technology          | React/Vite, Node.js/Express, TypeScript, MongoDB/Cosmos DB, Nginx, Azure, integrations                                              |
| Environments        | Production and Dev are currently routed through a shared Nginx VM; QA is supported by environment-specific deployment configuration |
| Out of scope        | UI field-by-field specifications, low-level API contracts, provider credentials and infrastructure-as-code runbooks                 |

## Contents

1. Executive Summary
  2. Scope, Assumptions and Design Principles
  3. Solution Architecture
  4. Application Components
  5. Business Workflow and Role Model
  6. Data Architecture
  7. External Integrations
  8. Security Architecture
  9. Deployment and Nginx Routing
  10. Operations, Monitoring and Resilience
  11. Non-Functional Requirements
  12. Risks and Recommendations
- Appendix A. API Domains and Environment Controls

# 1. Executive Summary

VMS is a role-based vendor lifecycle platform for initiating, screening, onboarding, validating, approving, integrating and servicing vendors. It supports domestic, international, government, credit-card, distribution and re-onboarding journeys.

The platform separates presentation, workflow logic, persistence and third-party integrations. The frontend is a React single-page application. The backend is a TypeScript/Express REST service structured around feature modules, controllers, services and repositories. MongoDB-compatible persistence is used for operational records, audit trails and user data.

**Current validated routing:** Nginx on the Azure VM serves Production at `/` and Dev at `/dev/`. It forwards Production API traffic to a local backend upstream on port 8080 and Dev API traffic to port 8081. HTTPS, SPA fallback and role/API routing are handled at the Nginx tier.

## Key architecture outcomes

- Separation of frontend delivery, API processing, data persistence and external integrations.
- Role-scoped workflow access for SPOC, AC, Manager, VM, VM Manager, Finance, Tax, Compliance, Business, Distributor, ASM, RSM and Administrator functions.
- Controlled integration adapters for Oracle Fusion/OIC, KYC providers, email/OTP and SAML identity federation.
- Auditability through workflow states, notifications, e-sign audit records and dedicated audit logs.

## 2. Scope, Assumptions and Design Principles

In scope

- Vendor initiation, vendor KYC, document proof validation, AC/TPI screening and approval workflow.
- Supplier and invoice integration lifecycle, including asynchronous retry status recording.
- Employee identity through OTP and enterprise SAML; vendor identity through OTP/KYC-specific flows.
- Operational dashboards, role-scoped queues, documents, notifications, chat and audit evidence.

### Design principles

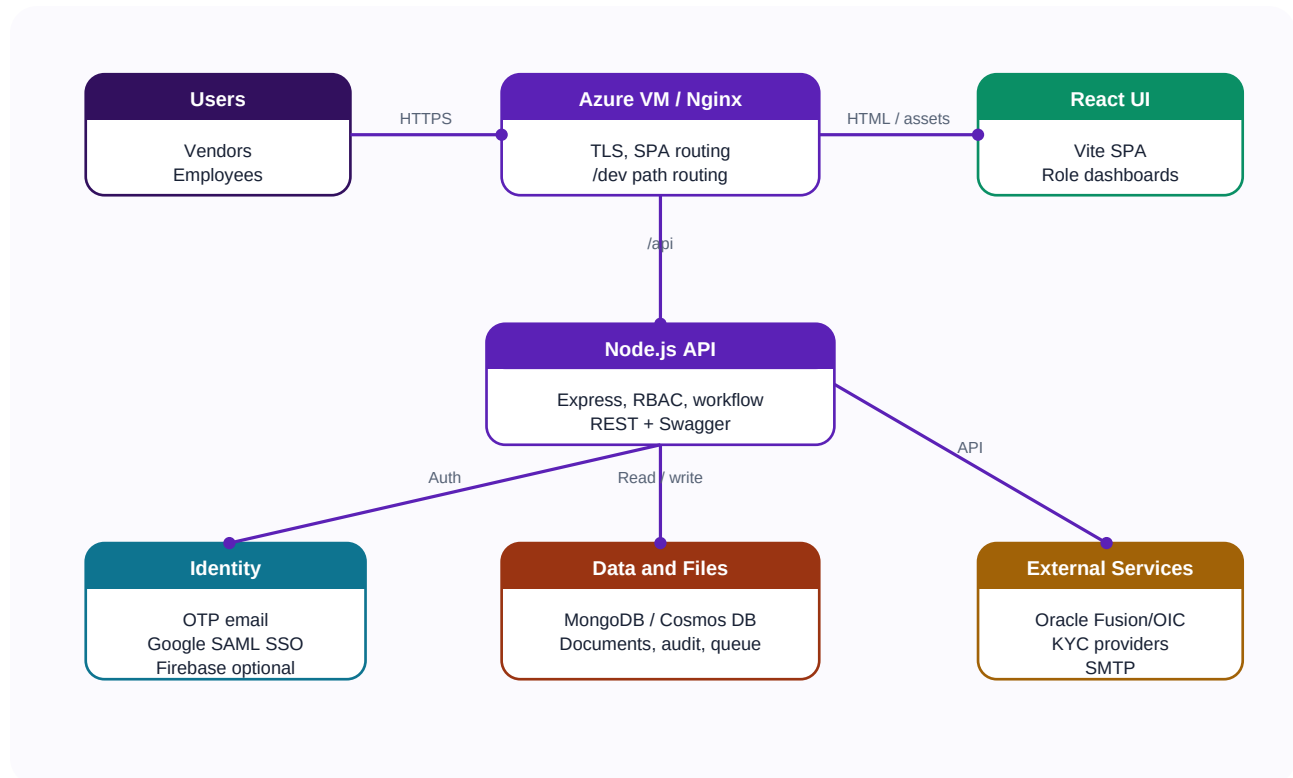
| Principle                 | Application                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Security by default       | Role-based authorization, JWT/session controls, input validation, rate limiting, security headers and encrypted/salted sensitive identifiers. |
| Workflow integrity        | State transitions and approvals are persisted, audited and surfaced to the relevant queue/dashboard.                                          |
| Provider isolation        | KYC and ERP integrations are accessed through dedicated service/adaptor layers and can be disabled or mocked by environment.                  |
| Environment isolation     | Dev, QA and Production use distinct URLs, configuration and database targets.                                                                 |
| Operational observability | Health endpoints, structured logs, API documentation, audit records and integration status fields support supportability.                     |

### Assumptions

The HLD reflects the repository as analysed and the Nginx behaviour observed on 07 September 2026. It should be updated after infrastructure, domain, identity provider or integration changes are approved.

### 3. Solution Architecture

The platform follows a layered web architecture. Nginx is the public reverse-proxy and static content entry point. The React SPA calls REST endpoints over the same protected transport. The backend owns business rules and coordinates persistence, notifications and third-party integrations.



#### Logical request path

| Step                           | Description                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1. Browser request             | Employee or vendor accesses the VMS URL over HTTPS.                                                          |
| 2. Edge routing                | Nginx redirects HTTP to HTTPS, serves SPA assets and routes API prefixes to the applicable backend.          |
| 3. Authentication              | OTP, token/session validation or SAML identity flow establishes the user context.                            |
| 4. Authorization               | Middleware validates role and endpoint permission before controller execution.                               |
| 5. Workflow service            | Feature service validates input, applies business state transition and creates audit/notification records.   |
| 6. Persistence and integration | Repositories write MongoDB/Cosmos-compatible records; adapters invoke configured external systems as needed. |

## 4. Application Components

### Frontend

| Component       | Responsibility                                                                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| React 18 + Vite | Single-page application build and delivery.                                                                                                  |
| React Router    | Role-aware routes for dashboards, initiation, approval, documents, invoices, vendor portal and administration.                               |
| Feature modules | Auth, Dashboard, Vendor, Domestic, International, Government, Credit Card, Distribution, Invoice, Notification, Chat, Roles and Permissions. |
| API client      | Token propagation, request handling, error normalisation, CORS-aware requests and download support.                                          |
| Swagger UI view | Authenticated/administrative API exploration where enabled.                                                                                  |

### Backend

| Layer                      | Responsibility                                                                                                          |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Express routes/controllers | HTTP contracts, validation wiring and response handling.                                                                |
| Feature services           | Business logic for workflows, approvals, KYC, vendor lifecycle, invoice and notifications.                              |
| Repositories/models        | Mongoose persistence access and collection-level domain models.                                                         |
| Middleware                 | Authentication, role authorization, SAML security, CSRF/security headers, rate limits, sanitisation and error handling. |
| Workers/queues             | Optional asynchronous processing for retries, notifications and integration tasks.                                      |
| Documentation              | Swagger/OpenAPI generation and API documentation endpoints.                                                             |

## 5. Business Workflow and Role Model

The workflow supports vendor categories with controlled routing rules. The exact approval chain depends on module type, business unit, TPI decision and role configuration.



### Primary roles

| Role group                           | Typical responsibilities                                                                                                               |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| SPOC                                 | Initiate vendor onboarding or standalone TPI screening, submit information, monitor assigned vendor status and respond to corrections. |
| AC Team                              | First-stage screening and TPI/due-diligence decisions; sends outcomes to SPOC.                                                         |
| Manager / Business / Business Unit   | Business review, hierarchy-driven routing, commercial and operational approval.                                                        |
| Tax / Compliance / Finance / Account | Tax review, compliance evidence, financial checks, invoice/payment related actions.                                                    |
| VM / VM Manager                      | Vendor management oversight, final workflow approvals and user/team administration as authorised.                                      |
| ASM / RSM / Distributor              | Distribution lifecycle review and channel-specific workflows.                                                                          |
| Administrator                        | System governance, selected user/role configuration, audit and administration functions.                                               |
| Vendor                               | Completes KYC, maintains profile/documents and views permitted service information.                                                    |

### State management

Onboarding records carry module type, workflow state, KYC status, approval history, assignment/hierarchy references and Fusion integration state. Key examples include SPOC review, AC screening, approval stages, rejection/correction states and Fusion integrated status. The system should treat state transitions as the authoritative source for queue/dashboard decisions.

## 6. Data Architecture

The backend uses Mongoose over a MongoDB-compatible data store. Azure Cosmos DB for MongoDB is used in deployed environments according to the supplied environment configuration. Collections are domain-oriented and include user, vendor, workflow, document, notification and audit data.

| Domain           | Representative records                                                 | Purpose                                                                    |
|------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Identity         | Users, OTPs, sessions, tokens, teams                                   | Employee/vendor access, role membership, session lifecycle and hierarchy.  |
| Vendor lifecycle | Vendor, vendor onboarding, vendor modifications, AC screening requests | Master profile, initiation payload, state, assignments, KYC and screening. |
| Evidence         | Documents, document reminders, e-sign requests/OTPs/audits             | Submitted proof, document expiry, signatures and evidence traceability.    |
| Operational      | Invoices, notifications, chats/messages, deboarding requests           | Business processing, communication and post-onboarding activities.         |
| Governance       | Audit logs, blacklist records, bulk-user-create log                    | Accountability, security controls and operational traceability.            |

### Sensitive data controls

- PAN-related fields are designed for encryption and hash-based lookup through PAN\_ENCRYPTION\_KEY and PAN\_HASH\_SECRET configuration.
- Bank, tax, identity and document fields are business-sensitive and should be encrypted/redacted in logs and restricted by role.
- No passwords, private keys or raw credentials are included in this HLD.



## 7. External Integrations

Integrations are configuration-driven. They must be enabled only in the intended environment and should use non-production credentials for Dev/QA.

| Integration                             | Direction                  | Purpose                                                                                                                | Key design control                                                                           |
|-----------------------------------------|----------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Oracle Fusion / OIC                     | Outbound                   | Supplier creation/update and invoice creation. The VMS records supplier identifiers, status, error and retry metadata. | Feature flag, endpoint configuration, retry/error capture and idempotency-aware workflow.    |
| TruthScreen                             | Outbound                   | PAN, GST, CIN, Udyam and bank verification through encrypted provider protocol.                                        | TRUTHSCREEN_ENABLED gate, HTTPS-only endpoint, timeout, cache and structured error category. |
| Gridlines                               | Outbound optional          | Bank and MCA/CIN lookup where configured.                                                                              | Credential gate and graceful failure if unavailable.                                         |
| SAML IdP / Google Workspace             | Inbound identity assertion | Enterprise SSO for authorised employees.                                                                               | Signed assertion validation, configured ACS/Entity ID and group-to-role mapping.             |
| SMTP / Microsoft Graph                  | Outbound                   | OTP, KYC links, notifications and workflow emails.                                                                     | Environment credentials and rate/recipient validation.                                       |
| Firebase optional                       | Identity / notifications   | Google sign-in or notification capabilities where configured.                                                          | Health status must be monitored; current readiness depends on credentials.                   |
| Object storage / MinIO or S3 compatible | Document storage           | Uploaded document storage when enabled.                                                                                | Bucket access control, signed access and retention policy.                                   |

### Integration safety

Provider calls must not be initiated during unit testing. The codebase supports disabled-provider and mock-oriented tests for response mapping, cryptography and validation. Production calls require approved credentials, consent and operational monitoring.

## 8. Security Architecture

Security is applied across browser, API, storage and integration boundaries.

| Control area       | Design                                                                                                                        |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Authentication     | OTP flows, JWT access/refresh tokens, sessions and enterprise SAML. Vendor-specific flows are separated from employee access. |
| Authorization      | Role-scoped routes and permission checks before business actions. Multiple portal roles can be represented for a user.        |
| Transport          | HTTPS at Nginx; HTTP redirected to HTTPS. Reverse proxy forwards X-Forwarded headers.                                         |
| Input protection   | Express validation, sanitisation, content/file validation, XSS controls and NoSQL injection protection.                       |
| Abuse prevention   | Rate limiting, slow-down controls and OTP attempt controls.                                                                   |
| Sensitive data     | Encrypted PAN fields/hashes, secure environment variables, redaction expectation and restricted document access.              |
| Auditability       | Audit logs, e-sign audit records, workflow decision fields and notification history.                                          |
| Runtime protection | Helmet headers, CORS origin controls, CSRF/session protections and secure cookie practices where enabled.                     |

**Required operations:** keep credentials in Azure App Service/VM secret stores or a managed vault; never commit .env files, OTPs, certificate private keys, database URIs or external API passwords.

## 9. Deployment and Nginx Routing

The checked Nginx server runs on an Azure VM. It terminates HTTPS and provides both Production and Dev paths. This enables one public IP/domain to host segregated frontend assets and backend upstreams.

| Route           | Frontend behaviour                                                 | Backend behaviour                                                                                          |
|-----------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| /               | Serves Production SPA from the Nginx web root with SPA fallback.   | /api/ proxies to Production backend upstream on 127.0.0.1:8080. /health and /api-docs/ are available.      |
| /dev/           | Serves Dev SPA using the /dev path and SPA fallback.               | /dev/api/ proxies to Dev backend upstream on 127.0.0.1:8081. /dev/health and /dev/api-docs/ are available. |
| HTTP port 80    | Redirects to HTTPS except ACME challenge.                          | N/A                                                                                                        |
| HTTPS port 443  | Primary public endpoint.                                           | Nginx reverse-proxy boundary.                                                                              |
| HTTPS port 8443 | Alternative Dev-oriented server block identified in configuration. | Dev backend upstream on 8081.                                                                              |

### Validated operational checks on 07 September 2026

- Nginx configuration syntax passed.
- Production and Dev SPA URLs returned HTTP 200 over HTTPS and had distinct content hashes.
- Production /health and Dev /dev/health returned HTTP 200 with database connection state connected.
- Production and Dev API documentation returned HTTP 200.
- OTP endpoint preflight requests returned HTTP 204 in both environments.

### Domain cutover

For a final domain, point DNS to the Nginx VM public IP, update server\_name and certificate configuration, and retain one canonical frontend/API origin in the application environment variables. Update SAML ACS and allowed redirect/IdP configuration only through the approved identity-management process.

## 10. Operations, Monitoring and Resilience

Operational design should distinguish user-visible health, application health, integration health and data recovery readiness.

| Area       | Current / expected control                                                                                                                                              |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Health     | Nginx serves the frontend and exposes health routing; Node /health reports database and selected service status.                                                        |
| Logs       | Nginx access/error logs and backend structured logging should be centrally retained with request IDs where available.                                                   |
| Monitoring | Alert on availability, 5xx rate, latency, authentication errors, queue/retry depth, database connectivity and integration failures.                                     |
| Backups    | Confirm Cosmos DB/Mongo backup mode, retention and point-in-time restore with the infrastructure owner.                                                                 |
| Recovery   | Document RTO and RPO as approved business commitments; do not infer contractual values solely from hosting technology.                                                  |
| Scaling    | Nginx VM is a single ingress point unless a load-balanced/multi-instance design is adopted. Backend and database scale strategies should be documented per environment. |
| Deployment | Build validation, API documentation generation, test gates and environment configuration promotion should be traceable through CI/CD.                                   |

### Operational watch item

The observed health output reported Firebase Admin as not ready. If Firebase/Google Sign-In is part of the production login experience, configure and validate Firebase Admin credentials. Enterprise SAML and OTP can remain separate flows, subject to their own configuration.

## 11. Non-Functional Requirements

| Category        | Design objective                                                 | Measure / validation                                                                             |
|-----------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Availability    | Protect core vendor workflow and user access.                    | HTTPS health checks, route monitoring, deployment rollback and database recovery evidence.       |
| Performance     | Fast SPA delivery and predictable API responses.                 | Static asset caching, gzip, Nginx proxy timeouts, API latency dashboards.                        |
| Scalability     | Support growth in users, onboarding volume and document traffic. | Horizontal backend plan, database capacity monitoring, queue workers and object storage scaling. |
| Security        | Protect employee, vendor, tax and financial data.                | RBAC tests, secret rotation, access reviews, pen-test/SAST and audit-log review.                 |
| Auditability    | Explain each material workflow decision.                         | Immutable/retained audit records, decision metadata and notification history.                    |
| Maintainability | Enable safe feature evolution across modules.                    | TypeScript build, automated tests, OpenAPI generation, code review and versioned deployments.    |
| Compliance      | Support data minimisation and retention controls.                | Approved data classification, retention schedule, provider agreements and access controls.       |

## 12. Risks and Recommendations

| Priority | Risk / gap                                                                            | Recommendation                                                                                                                                |
|----------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| High     | Single Nginx VM is a potential ingress single point of failure.                       | For production criticality, assess Azure Load Balancer/Application Gateway/Front Door and a second ingress instance.                          |
| High     | Secrets exist across multiple integrations and environments.                          | Move/retain all secrets in managed secret storage, enforce rotation, and remove credentials from shared terminals/chats and repository files. |
| High     | RTO/RPO not evidenced by this repository analysis.                                    | Obtain approved BCP values and validate Cosmos DB backup/restore procedure.                                                                   |
| Medium   | Provider integrations may have cost and data-processing implications.                 | Use feature flags, mock contracts in Dev/QA and audited production enablement.                                                                |
| Medium   | Nginx domain configuration currently uses generic server_name in the observed config. | Before domain go-live, set canonical hostnames, strict redirect rules, certificates, HSTS policy and CORS origin values.                      |
| Medium   | Firebase Admin reported not ready.                                                    | Either configure supported Firebase credentials or remove/disable unused Firebase login functionality from user-facing paths.                 |
| Medium   | Document validation requires continual accuracy testing.                              | Keep representative consented document fixtures and regression tests for OCR/CIN/PAN/GST/bank evidence.                                       |

### Next steps

- Approve target domain naming and certificate ownership.
- Document Dev, QA and Production configuration baselines without embedding secrets.
- Run end-to-end workflow tests against an isolated test database and mocked external providers.
- Formalise backup restore exercise, RTO and RPO, and operational escalation path.
- Review access roles and SAML group-to-role mappings with the identity owner.

# Appendix A. API Domains and Environment Controls

Representative API domains

| Domain                   | Representative functions                                                                    |
|--------------------------|---------------------------------------------------------------------------------------------|
| Authentication           | OTP request/verify, token refresh, SAML login/ACS/metadata, vendor authentication.          |
| Vendor lifecycle         | Initiation, onboarding, KYC completion, correction, re-onboarding, approval, vendor master. |
| Screening and validation | AC/TPI screening, PAN, GST, CIN, MSME/Udyam, bank and document validation.                  |
| Commercial               | Invoices, payments, security deposits and Oracle Fusion synchronisation.                    |
| Collaboration            | Notifications, employee/vendor chat, documents, reminders and e-sign.                       |
| Governance               | Roles, teams, user administration, dashboard metrics, audit logs and deboarding.            |

## Environment configuration categories

| Category                  | Examples (names only)                                                    |
|---------------------------|--------------------------------------------------------------------------|
| URLs and origins          | APP_BASE_URL, FRONTEND_URL, CORS_ORIGINS, VENDOR_PORTAL_URL.             |
| Database                  | MONGODB_URI or environment-specific Mongo connection setting.            |
| Identity                  | JWT secrets, SESSION_SECRET, SAML IdP/SP metadata and Firebase settings. |
| Sensitive-data protection | PAN_ENCRYPTION_KEY, PAN_HASH_SECRET.                                     |
| External integrations     | ORACLE_FUSION_*, TRUTHSCREEN_*, GRIDLINES_*, SMTP / Graph settings.      |
| Runtime                   | NODE_ENV, ports, logging, queue/cache and worker controls.               |

## HLD acceptance criteria

- Architecture has been reviewed by application, infrastructure, security and integration owners.
- Environment-specific DNS, TLS, CORS and SAML values are documented separately as controlled configuration.
- RTO/RPO, backup evidence and monitoring ownership are approved before production handover.
- No secret values are present in this HLD.